

FIN-220 Hands-On Machine Learning

HW 2

Due on October 6, 2025

In my Github:

https://github.com/McSavage/FIN220/blob/main/assignments/HW2_Question123.ipynb

The following exercises are based on the housing dataset.

1. Try a support vector machine regressor (`sklearn.svm.SVR`) with various hyperparameters, such as `kernel="linear"` (with various values for the `C` hyperparameter) or `kernel="rbf"` (with various values for the `C` and `gamma` hyperparameters). Note that support vector machines don't scale well to large datasets, so you should probably train your model on just the first 5,000 instances of the training set and use only 3-fold cross-validation, or else it will take hours. We will discuss hyperparameters in Chapter 5.
5. How does the best SVR predictor perform?

Starting with `preprocessing = ColumnTransformer(`

`from the 02_end_to_end_etc., notebook...`

```
from sklearn.svm import SVR
from sklearn.model_selection import cross_val_score

# Let's start with a basic SVR with RBF kernel using the full dataset
svr_reg = make_pipeline(preprocessing, SVR(kernel='rbf', C=100, gamma='scale', epsilon=0.1))
svr_reg.fit(housing, housing_labels)
```

I get:

SVR RMSE: 93,086

VS the

Linear Regression RMSE: 68,648

Improvement: -35.6% - Worse

Using:

```
from sklearn.model_selection import cross_val_score
```

with the kernels 'linear', 'rbf', and 'poly' -

linear returns the smallest RMSE

LINEAR SVR - RMSE: 78,666 ($\pm 3,987$) - Time: 10.1s

RBF SVR - RMSE: 98,387 (± 847) - Time: 17.0s

POLY SVR - RMSE: 216,572 ($\pm 149,645$) - Time: 10.9s

For comparison, Linear Regression RMSE: 68,648

Using:

```
from sklearn.model_selection import GridSearchCV
```

specifying the 'linear' model;

```
param_grid = {
    'svr__C': [10, 100, 1000],
    'svr__gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
    'svr__epsilon': [0.01, 0.1, 0.2]
}

svr_pipeline = make_pipeline(preprocessing, SVR(kernel='linear'))

# Using 3-fold CV on full dataset since training time is acceptable
grid_search = GridSearchCV(
    svr_pipeline,
    param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error',
    verbose=1,
    n_jobs=-1 # Use all available cores
)

grid_search.fit(housing, housing_labels)
```

Fit to the entire housing set (16512 rows) took less than 2 minutes on my desktop.

I did try with the first 5000 and got similar results

Best Parameter found:

```
{'svr__C': 1000, 'svr__epsilon': 0.01, 'svr__gamma': 'scale'}
```

Best cross-validation RMSE: 71,718 k – 3.7% worse than the simple linear model

A lot of the cleanup/optimization is done in data pipeline

2. Try replacing the GridSearchCV with a RandomizedSearchCV

Using the 'rbf' kernel;

```
3. param_distributions = {
4.     'svr__C': loguniform(1, 1000),          # Log-uniform distribution from 1 to 1000
5.     'svr__gamma': loguniform(0.0001, 1),    # Log-uniform from 0.0001 to 1
6.     'svr__epsilon': uniform(0.01, 0.5),     # Uniform from 0.01 to 0.51
7. }
8.
9. # Create SVR pipeline (you can change kernel here)
10. svr_pipeline_random = make_pipeline(preprocessing, SVR(kernel='rbf'))
11.
12. # RandomizedSearchCV with budget control
13. start_time = time.time()
14.
15. random_search = RandomizedSearchCV(
16.     svr_pipeline_random,
17.     param_distributions=param_distributions,
18.     n_iter=50, # Number of parameter settings to sample
19.     cv=3,
20.     scoring='neg_root_mean_squared_error',
21.     verbose=1,
22.     n_jobs=-1,
23.     random_state=42 # For reproducibility
24. )
25.
26. random_search.fit(housing, housing_labels)
```

RandomizedSearchCV took about 3 minutes to run on the housing data.

RandomizedSearchCV RMSE: 73,656 – 7.3% worse than the simple linear model

I think a lot of the cleanup on the model is done in the pipeline.

3. Try adding a SelectFromModel transformer in the preparation pipeline to select only the most important attributes

```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestRegressor
selector_pipeline = Pipeline([
    ('preprocessing', preprocessing),
    ('selector', SelectFromModel(RandomForestRegressor(random_state=42),
                                threshold=0.005)), # min feature importance
    ('svr', SVR(C=random_search.best_params_["svr__C"],
               gamma=random_search.best_params_["svr__gamma"],
               kernel="linear")),
])
```

Using Select from Model Random Search RMSE is 78,2445